Prasita Mukherjee — Research Statement

My research focuses on broadening the scope of hard problems in the domain of formal methods and programming languages by providing learning-based solutions.

Research Background: In my academic career, I have worked on several research projects, primarily in the domains of formal methods and programming languages. In the next sections, I describe three *research directions* I have explored and proposed a solution by incorporating a learning algorithm with symbolic approach. I conclude with other relevant research areas on which I have worked and my contributions to them.

Can Learning Algorithms classify whether a system satisfies an LTL formula? LTL model checking is known to be a computationally hard problem (PSPACE Complete [1]), and traditional model checkers prove costly as they suffer from the state space explosion problem that prevents model checking from being applied to systems and specifications with increasing complexity. To bridge this gap and make model checking more accessible, we designed a Graph Representation Learning framework **OCTAL** [2] that takes advantage of the natural graph structure of the automaton and expression tree structure of LTL to build a unified graph structure, and learn the structure and semantics of the system and specification through message passing. We performed rigorous experiments on OCTAL, and obtained up to $\sim 95\%$ accuracy on cases where the system and specifications are equivalent. This is the first line of work that demonstrates whether Graph Neural Networks [3] can detect if a system satisfies a specification and can therefore be incorporated in a model checker to filter out candidates in a lower risk environment; thus gaining efficiency.

Can LLMs synthesize a heap manipulating program given its specification? Synthesizing a program from the underlying specification describing its full functional correctness is an undecidable problem [4] in the general case. Pure symbolic approaches [5] are not scalable due to the search space, and/or cater to specifications that describe part of its behavior — for example, just the memory layout and not the underlying values. With the advent of Large Language Models and them being deployed for complex tasks owing to their large search space, we propose a novel, general purpose synthesizer for C programs equipped with machine-checked proofs of correctness using the Verified Software Toolchain (VST) [6]. To do so, our tool SynVer employs two Large Language Models (LLMs): the first generates candidate programs from user-provided specifications, and the second helps automatically construct formal proofs of their correctness in the Rocq proof assistant. To facilitate verification, SynVer places a set of syntactic restrictions on candidate programs that make them amenable to automated reasoning. SynVer uses a hybrid verification strategy that combines symbolic reasoning with LLM-powered proof generation to discharge proof obligations that the symbolic engine cannot handle on its own. We demonstrate the applicability of SynVer using a diverse set of benchmarks drawn from the program synthesis and verification literature, and conclude that SynVer is generalizable to different categories of synthesis, with respect to full functional specifications of a program.

Can LLMs correctly infer a Separation Logic loop invariant? In SynVer, we demonstrated the applicability of LLMs to generate and prove a program correct with respect to the given specification. However, the caveat of SynVer is that we require an expert to provide the complete specification, which is oftentimes more complex than the program synthesized. Further, to reduce the verification complexity, one of the syntactic bias placed by SynVer require the programs generated to be recursive. However, recursion may not be the natural choice due to efficiency. To bridge this gap, we propose a novel Specification Inference and

Property Based Testing [7] framework SMEAGOL. Given the pre and postcondition of a heap manipulating program with loop(s), SMEAGOL employs an LLM to generate an invariant for a loop. The invariant argues about the heap layout as well as the contents of the data structures in the presence of a set of recursive predicates and functions, therefore involves operators like the *magic wand* [8] which elegantly capture the layout desired but is undecidable in general [9, 10]. Therefore, the second phase of SMEAGOL is a novel property based testing algorithm that tests specifications written with the *magic wand* operator. We evaluate our approach on a set of programs operating over rich data structures, including doubly linked lists and binary trees. Our experiments demonstrate that our testing technique is able to effectively discard invalid invariants proposed by an LLM, enabling the end-to-end pipeline to effectively propose correct invariants in a majority of cases, across a range of models and reasoning levels.

Other relevant works and Summary *Automated reasoning* seeks to provide correct solutions to questions framed in logic with no user assistance. This body of research is very challenging and appealing as questions in first, or higher order logic are undecidable in general [11]. Hence, providing provably correct solutions and identifying decidable fragments of the problem at hand, would make it easier, especially for end users. My research in this domain has been on three projects: First, analyzing the equivalence of snippets of a programming language. Second, analyzing the feasibility of shared memory data structures in a distributed system setting [12], and third, relational verification of programs using E-Graphs [13]. My research in the first two projects was to encode the execution semantics of the programming language and data structures into an SMTLIB [14] format, query an SMT solver [15, 16] for analysis, identify the decidable fragment of the language and perform experimental analysis to draw conclusions for the respective problems. The third project aims to verify if two programs are observationally equivalent, by constructing a product program alignment using E-Graphs [17]. We demonstrate that the product programs are relatively easier to verify, requiring simpler invariants. My contribution was to come up with manual loop invariants where automated inference failed, and prove them in VST [6], along with the demonstration that the invariants although complex enough to require interactive theorem proving, were simpler compared to other alignments.

To summarize, my primary research focus has been integrating learning models with pure symbolic approaches to solve for scenarios that are otherwise prohibitive due to the non-trivial search space. My research spans across fully automatic domains like LTL Model Checking, to writing interactive proofs inside theorem provers like Rocq [18], and lightweight approaches like Property Based Testing to validate solutions generated by Large Language Models. The span of my work also involves developing manual proofs, writing full functional specifications in Separation Logic [8], and mapping custom data structures and queries in SMTLIB. Over the course of my PhD., I have obtained rigorous experience in analyzing and scoping solutions to intractable, or undecidable problems in Formal Methods and Programming Languages by identifying the concise way of combining learning algorithms with symbolic techniques. My hands-on experience in working with a spectrum of verification tools has enhanced my ability to develop the concise symbolic approach to push forward the state of the art. To conclude, I have strong foundations in Formal Methods research, and have advanced state of the art across three different domains, along with rigorous hands on experience in working with a spectrum of verification tools that make me a strong candidate in this field.

References

[1] Christer Backstrom and Peter Jonsson. "All PSPACE-complete planning problems are equal but some are more equal than others". *Proceedings of the International Symposium on Combinatorial Search*. 2011.

[2] Prasita Mukherjee, Haoteng Yin, Susheel Suresh, and Tiark Rompf. "OCTAL: Graph Representation Learning for LTL Model Checking". *CoRR* (2022). DOI: 10.48550/arXiv.2207.11649. arXiv: 2207.11649. URL: https://doi.org/10.48550/arXiv.2207.11649.

[3] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. "A comprehensive survey on graph neural networks". *IEEE transactions on neural networks and learning systems* (2020).

[4] Alfred Tarski, Andrzej Mostowski, and Raphael Mitchel Robinson. *Undecidable theories*. Elsevier, 1953.

[5] Nadia Polikarpova and Ilya Sergey. "Structuring the synthesis of heap-manipulating programs". *Proceedings of the ACM on Programming Languages* (2019).

[6] Andrew W Appel. "Verified Software Toolchain: (Invited Talk)". *European Symposium on Programming*. 2011.

[7] George Fink and Matt Bishop. "Property-based testing: a new approach to testing for assurance". *ACM SIGSOFT Software Engineering Notes* (1997).

[8] John C Reynolds. "Separation logic: A logic for shared mutable data structures". *Proceedings 17th annual IEEE symposium on logic in computer science*. 2002.

[9] Rémi Brochenin, Stéphane Demri, and Etienne Lozes. "On the almighty wand". *Information and Computation* (2012).

[10] Jens Pagel and Florian Zuleger. "Strong-separation logic". *ACM Transactions on Programming Languages and Systems (TOPLAS)* (2022).

[11] Ian Pratt-Hartmann. *Fragments of first-order logic*. Oxford University Press, 2023.

[12] Kartik Nagar, Prasita Mukherjee, and Suresh Jagannathan. "Semantics, Specification, and Bounded Verification of Concurrent Libraries in Replicated Systems". *Computer Aided Verification - 32nd International Conference, CAV 2020, Los Angeles, CA, USA, July 21-24, 2020, Proceedings, Part I*. Ed. by Shuvendu K. Lahiri and Chao Wang. Lecture Notes in Computer Science. Springer, 2020. DOI: 10.1007/978-3-030-53288-8_13. URL: https://doi.org/10.1007/978-3-030-53288-8_13.

[13] Robert Dickerson, Prasita Mukherjee, and Benjamin Delaware. "KestRel: Relational Verification using E-Graphs for Program Alignment". *Proc. ACM Program. Lang.* (Apr. 2025). DOI: 10.1145/3720474. URL: https://doi.org/10.1145/3720474.

[14] Clark Barrett, Aaron Stump, Cesare Tinelli, et al. "The smt-lib standard: Version 2.0". *Proceedings of the 8th international workshop on satisfiability modulo theories (Edinburgh, UK)*. 2010.

[15] Leonardo De Moura and Nikolaj Bjørner. "Z3: An efficient SMT solver". *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. 2008.

[16] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. "cvc5: A versatile and industrial-strength SMT solver". *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. 2022.

[17] Ross Tate, Michael Stepp, Zachary Tatlock, and Sorin Lerner. "Equality saturation: a new approach to optimization". *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 2009.

[18] Christine Paulin-Mohring. "Introduction to the Coq proof-assistant for practical software verification". *LASER Summer School on Software Engineering*. Springer, 2011.